# Unit: a small dependent type checker with segments as modules

24 October 2026

**Abstract**

Unit is a prototype proof checker for a dependent type theory with Π, Σ, record types (tuples with named fields), data types given by constructors, an identity type with its eliminator, and a hierarchy of cumulative universes indexed by level expressions. Its module system develops the theory mechanism already implemented in Half [7], in the tradition of Automath segments and the implementation described in [13]: a module is a list of declarations, its parameters are the declarations without a body, and an instance only binds the parameters, without copying the body. A record type is written like a module, as a list of declarations, but it is a type and not a module; its elements are tuples of values for the declarations without a body. The prototype has been used to formalise a model of univalent type theory over posets and simplicial sets, results of synthetic algebraic geometry, synthetic Stone duality, and the cohomology of compact Hausdorff spaces in synthetic Stone duality. This note describes the type theory, the module system, and the choices made to keep checking predictable and fast. We also propose an organisation of checking around a list of conversion problems generated without unfolding definitions. Pairs settled by strict equality are discarded; the remaining pairs expose the work of conversion and provide the basis for explaining failures. We distinguish this proposal from what the current implementation already provides. The LLM-assisted formalisation of substantial examples in synthetic algebraic geometry provides a setting in which to investigate this proposal and compare the checker's work with the informal arguments; the formalisation of cohomology in synthetic Stone duality is presented as a case study. The name Unit refers to the units, the declarations of which every file, segment and local block is a list.

## Contents

**1 Overview** **2**

**2 Checking a proof through its conversion problems** **3**
2.1 The organisation of the argument . . . . . 3
2.2 Generate, discard, analyse . . . . . 3
2.3 Explaining a failed check . . . . . 4
2.4 Studying mathematical examples with LLM assistance . . . . . 4
2.5 Scope of the proposal . . . . . 5

**3 The type theory** **5**
3.1 Terms . . . . . 5
3.2 Universes and levels . . . . . 7
3.3 Definitional equality . . . . . 8

**4 The module system** **9**
4.1 Segments . . . 9
4.2 Instances share . . . 9
4.3 Parameters stated once . . . 10
4.4 Records . . . 10

**5 The checker** **11**
5.1 Syntax after resolution . . . 11
5.2 Values . . . 12
5.3 Conversion . . . 13
5.4 Bidirectional checking . . . 13
5.5 Units and segments . . . 14

**6 Notation read before checking** **15**
6.1 Omitted arguments . . . 15
6.2 Equational chains . . . 15
6.3 Scoping . . . 15

**7 A case study: cohomology in synthetic Stone duality** **16**

# 1 Overview

A file is a list of *units*. A unit is a declaration of one of the following kinds:

| | |
|---|---|
| `x (tel) : A ;` | a parameter (at the top level: a primitive notion) |
| `x (tel) : A = e ;` | a definition |
| `data D (tel) =` $c_1$ `(tel`$_1$`) \| ...;` | a labelled sum (a definition) |
| `level a b ;` | level variables |
| `constraint a < b ;` | constraints between levels |
| `module M (tel) { units } ;` | a segment |
| `module N = M[args] ;` | an instance of a segment |
| `record R (tel) { units } ;` | a record type |
| `opaque f ; transparent f ;` | the unfolding strategy |

(The scoping declarations `open` and `import` are read before checking; see Section 6.) Checking a list of units threads a *context of units*: each unit is checked in the context of the units before it, and is then added to it. A definition stays a definition in the context: its body is available for computation. The body of a `let` is a list of units as well, but local blocks cannot contain every kind of unit: declarations whose identities must survive evaluation are excluded (Section 3.1). The scope is sequential: a definition sees only the units before it, never itself or a later one, so ordinary definitions cannot form cycles. Recursion comes only from the eliminators of labelled sums (Section 3).

Two design principles run through the system. First, *nothing is guessed*: there is no unification, and the few arguments that may be omitted are found by a deterministic matching procedure (Section 6.1). Second, *nothing is copied*: a definition has one identity wherever it is used, and two uses are compared by that identity before anything is unfolded (Section 3.3).

The module mechanism has a direct predecessor in Half, developed by Thierry Coquand and implemented by Dan Synek, as described in Cederquist's thesis [7]. Its theories are lists of declarations, with parameters and local definitions; the thesis explicitly relates the theory mechanism

to PVS. This work predates Agda and was part of the background from which Agda developed. The present note revisits this mechanism and its interaction with conversion; it does not claim the invention of parameterised modules.

## 2 Checking a proof through its conversion problems

### 2.1 The organisation of the argument

Checking should follow the organisation of a mathematical proof. A construction is checked under its assumptions; an application is checked by verifying that its arguments satisfy those assumptions. Two uses of the same construction should be recognisable without going back through its definition. A segment makes this discipline explicit for a whole development: check its declarations once, then check the arguments of each instance.

This is also a guide to understanding checking time. When an informally routine step requires substantial computation, we should examine whether the representation or the checker has lost an abstraction used in the argument. This is a diagnostic principle, not a complexity bound: some proofs deliberately require substantial computation, and an implementation can also repeat work unnecessarily.

### 2.2 Generate, discard, analyse

The view of type checking as generating a list of conversion problems goes back to de Bruijn. The specific proposal here is to generate that list *without unfolding definitions*, and then discard pairs which are strictly equal before analysing the remaining conversion problems. The proposal concerns checking explicit terms, after scope resolution and the filling of omitted arguments. Level inequalities and other structural side conditions remain separate obligations.

Write the generated list schematically as

$$\mathcal{C}(\Gamma, t, A) = [\,(\Gamma_i, u_i, v_i, \ell_i)\,]_i,$$

where $\Gamma_i$ is the local context and $\ell_i$ records the source location and the reason for the comparison. An entry asks whether $u_i$ and $v_i$ are convertible in $\Gamma_i$. Definitions in these expressions retain their declaration identities and environments. Writing $=_{\mathrm{s}}$ for strict equality of this representation, the residual list is the sublist

$$\mathcal{C}_{\mathrm{res}} = [\,(\Gamma_i, u_i, v_i, \ell_i) \in \mathcal{C} \mid u_i \neq_{\mathrm{s}} v_i\,].$$

Conversion analyses this residual list. Discarding a strictly equal pair is a successful check, not an omission of a proof obligation. Conversely, membership in the residual list is not an error: it identifies an equality which needs further analysis.

Here strict equality means structural identity of the represented expressions, respecting the representation of bound variables, and without consulting the bodies of named definitions. In particular, references to the same declaration at strictly equal environments are strictly equal. It is not merely equality of the names printed for the user, and it is not full definitional equality. Administrative operations such as extending an environment must be distinguished from unfolding a named definition. A complete account must specify the representation and the structural comparison precisely.

For example, suppose $f : (x : A) \to B(x)$ and the inferred type of $a$ is $A'$. Checking $f\,a$ against $C$ gives the obligations

$$A' \equiv A, \qquad B(a) \equiv C,$$

in addition to those for the subterms. Their expressions should retain the names occurring in the argument. In the segment example of Section 4, the two aliases for `Mon[N, zero, plus].twice` refer to the same declaration at the same environment: their comparison is discarded without inspecting `twice`.

### 2.3 Explaining a failed check

The residual list should be intelligible as part of the mathematical argument. Each entry retains the original two expressions, its context, and its origin. If the first obligation in the application example fails, the diagnostic can say that the argument has type $A'$ whereas the parameter requires $A$, and display the unresolved comparison $A' \equiv A$ with both definitions still folded. Conversion may then provide a more detailed explanation, tied to this original obligation. The diagnostic should not replace the user's expressions by a large expansion merely because the conversion algorithm inspected it.

The same information can explain an expensive check: identify the original obligation being analysed and, when useful, the definitions whose unfolding it requires. This makes it possible to relate the work of the checker to a particular step of the proof. A timeout must be reported as an unfinished comparison, not as evidence of non-convertibility.

### 2.4 Studying mathematical examples with LLM assistance

Interaction between a mathematician and a large language model (LLM) makes it practical in this project to investigate these questions on substantial mathematical developments. The LLM helps write and revise formal proofs, inspect the implementation, and explore alternative representations. The checker determines whether the resulting formal terms are accepted. This allows a close iteration between an informal argument, its formal expression, and the work required to check it. Section 7 reports one such iteration on a complete paper.

The examples come from *A foundation for synthetic algebraic geometry* [10] and *Projective space in synthetic algebraic geometry* [11]. The companion note *Synthetic algebraic geometry in Unit: what has been formalised* records the results already checked and their hypotheses. Two examples suggest concrete comparisons. For boundedness of maps $\operatorname{Spec} A \to \mathbb{N}$ ([10], Theorem 3.3.3), both the argument through eventually vanishing sequences and an argument using the polynomial representing $(x, t) \mapsto t^{f(x)}$ have been formalised. These give two ways to organise the same mathematical result, whose intermediate constructions and checking costs can be compared.

The classification of line bundles on projective space gives a different test. The formal development separates the algebraic result of Horrocks, triviality on the affine charts of $\mathbb{P}^1$, the transition function on their overlap, and the passage to $\mathbb{P}^n$ using projective lines ([11], Proposition 4.6 and Section 7). One can ask whether using a result at each stage requires the checker to reopen the constructions from the preceding stages, and which equalities actually require their definitions. The module and definition mechanisms should preserve these mathematical boundaries.

For such a case study, the useful evidence is a correspondence between a step of the informal proof, the formal declarations expressing it, and the residual conversion problems it generates. We can record which problems disappear by strict equality, which require unfolding, and which dominate checking time. An unexpectedly expensive problem can then be examined with LLM assistance: does it expose a missing mathematical lemma, a representation that loses sharing, or repeated work in the checker? A proposed change is checked again, keeping track of the theorem and its hypotheses as well as the change in conversion work. The same examples can test whether a diagnostic identifies the mathematical mismatch in terms the reader recognises.

The practical contribution of LLM assistance here is the ability to carry out and revise these investigations on real proofs. The completed formalisations provide the material; the systematic analysis of their residual conversion problems is work to be done. Total checking times alone do not establish the correspondence with informal proof checking.

### 2.5 Scope of the proposal

The current prototype preserves definition references in values, tries strict comparison before conversion, and retains folded expressions for diagnostics. It still interleaves structural type checking with conversion rather than first producing the complete residual list. There is a further issue in making the generation phase literally free of unfolding: the current checker can unfold a type to expose its outer form, for example a product when checking an application. Preserving the unfolded type's original expression for a diagnostic does not remove that computation.

Thus the proposed separation requires an explicit account of how such type shapes are obtained, whether from annotations, stored structural information, or additional obligations. Its correctness must also account for dependencies between obligations: later checks may rely on types or contexts whose validity is still pending. The intended result is that, when the structural checks succeed, the declaration is accepted exactly when its generated obligations are discharged. This section states the design to be developed, not a completed correctness theorem or a claim that the implementation already generates every obligation without unfolding.

## 3 The type theory

### 3.1 Terms

The terms are those of a dependent type theory with definitions:

| | |
|---|---|
| `U l` | the universe at level `l` |
| `(x : A) -> B`, `\ x -> t`, `t u` | dependent functions |
| `(x : A) * B`, `(t, u)`, `t.1`, `t.2` | dependent pairs |
| `R[args]`, `new R[args]`, `t.l`, `split { R` $x_1$`...-> t }` | records |
| `sig { l`$_1$ `: A`$_1$ `; ...}`, `struct { l`$_1$ `= t`$_1$ `; ...}` | anonymous records |
| `c` $t_1$ `...`$t_n$, `split { c`$_1$ $x_1$`...-> t`$_1$ `; ...}` | constructors and case analysis |
| `Id A a b`, `refl a`, `idJ A a C d b p` | the identity type |
| `[a] A`, `<a> t`, `t @l` | products over levels |
| `[a < b] A` | products over constraints |
| `let { units } in t` | local definitions |
| `M[args].x` | a member of a segment instance |

**Labelled sums.** A `data` declaration introduces a type with named constructors, each with a telescope of arguments. A function out of a sum is written `split`, a list of branches, one per constructor; `split` is not a term on its own but is checked against a function type $(x : D) \to P\,x$ whose domain is a sum. Recursion is not by name: Unit has *primitive recursion*, through the eliminators of the sums. A branch may bind, after its pattern variables, one *recursive result* for each recursive field of the constructor:

```
data N = zero | suc (n : N) ;
plus (n : N) : N -> N = split { zero -> n ; suc m | r -> suc r } ;
```

For a field $m : D$ the result has type $P\,m$; for a function field $f : (y : \Psi) \to D$, as in `sup (A : U l) (f : A -> M)`, it has type $(y : \Psi) \to P\,(f\,y)$. The motive $P$ contains everything after the argument split, so the result is a function when the split is followed by further arguments. The arguments before the split are fixed: in `plus`, `r` is `plus n m`, of type `N`. A nested split has its own results: in Ackermann's function

```
ack : N -> N -> N = split {
  zero -> \ n -> suc n ;
  suc m | r -> split { zero -> r (suc zero) ; suc n | s -> r s } } ;
```

`s` is `ack (suc m) n` and `r` is `ack m`. This is primitive recursion at higher types, as in Hilbert's *On the infinite* [9] and Gödel's system T: since a recursive result may be a function, Ackermann's function is definable. Mutual recursion is written as one recursion computing a pair, and course-of-values recursion likewise. The list of results may be omitted when a branch does not use them; when present it is complete, and `_` names an unused result. This is the *inductive case expression* of Burstall [4], where a variable $v$ of the type of the argument, bound in a pattern, comes with a variable $\$v$ for the value of the whole expression at $v$, so that termination follows from the scope rules; his `plus` is, up to the order of the arguments, the one above, and he already uses results of higher type.

The sums are restricted so that these eliminators are the usual ones. A data type sees its own name only in its constructor fields, never in its parameters or type, so there are no mutual sums. A field mentions the type only as $D\,\vec{p}$ or $(y : \Psi) \to D\,\vec{p}$, where $\Psi$ does not mention $D$ and $\vec{p}$ are exactly the parameters of $D$: the sums are strictly positive, with uniform parameters, and there are no nested or indexed inductive families. Every recursive computation then starts from an eliminator whose recursive results the checker constructs, on a recursive field of a constructor value: every recursion is structural.

A `split` or a `data` may occur only at a *definition position*: as the body of a definition, after its $\lambda$s, or inside the branches of such a `split`. Units defining data types or recursive functions must additionally be outside `let` blocks (Section 3.1). Elsewhere, for instance as the argument of a function, it is rejected. A `split` is then written back as its definition applied to the values of the variables on its path, a term in the current scope, provided its recursive results are those that path determines; otherwise (for instance under the abstract results of a branch being compared) it is not written back.

**Local units and read-back.** A `let` block must not introduce a data type, a recursive definition, a module, or any other notion whose identity is used for comparison by name. In particular this includes records compared by declaration identity. The restriction also applies inside declarations nested in the block. A `where` block is subject to the same restriction, since it is read as a `let`.

This is a purely syntactic restriction, checked before type checking. A traversal of the syntax records whether it is inside a local block and rejects the prohibited declaration forms there. It requires neither evaluation nor conversion, and does not try to determine whether a particular value would actually escape the block. The read-back argument explains the restriction; it is not part of the check itself.

The reason is that evaluation can return a value which still refers to such a declaration after its local scope has been left. Read-back must then express that value in the surrounding scope, where the declaration has no available name. For example, returning a locally declared data type leaves a type identity with no name in that scope; returning a member of a local module can similarly leave a reference to its local declaration and environment. Recreating a declaration with the same

body would introduce a new identity, so it does not in general recover the same value. Nor can unfolding be relied upon to eliminate recursive definitions. The required invariant is that every declaration identity retained by an evaluated value has a valid reference when that value is read back. Ordinary local abbreviations are compatible with this discipline only insofar as their local names can be eliminated; it is the retention of an identity, rather than the mere presence of a name in the source, that causes the problem.

**The identity type.** `Id A a b` is a primitive type with `refl a : Id A a a` and the eliminator

$$\frac{C : (y : A) \to \mathsf{Id}\, A\, a\, y \to \mathsf{U}\, l \qquad d : C\, a\, (\mathsf{refl}\, a) \qquad p : \mathsf{Id}\, A\, a\, b}{\mathsf{idJ}\, A\, a\, C\, d\, b\, p : C\, b\, p}$$

with the computation rule $\mathsf{idJ}\, A\, a\, C\, d\, a\, (\mathsf{refl}\, a) = d$. Since there is no pattern matching on `refl`, there is no way to derive uniqueness of identity proofs. With strictly positive sums and recursion only through their eliminators, the theory is meant to be compatible with univalence, which a development may assume as a parameter (Section 4.3).

### 3.2 Universes and levels

The universes follow the system of [2]. Levels are expressions

$$l \;::=\; a \;\mid\; l^{+} \;\mid\; l \vee m$$

in level variables $a$, a successor and a maximum. There is no level 0 for the user. Level variables are declared by `level a`, and may be constrained by `constraint` units ($l < m$, $l \leq m$, $l = m$).

The universes are *à la Russell*: an element of `U l` is itself a type. The rules are

- $\mathsf{U}\, l : \mathsf{U}\, l^{+}$;
- $\Pi$, $\Sigma$, records and $\mathsf{Id}$ lie in the maximum of the levels of their components;
- cumulativity: if $A : \mathsf{U}\, l$ and $l \leq m$ then $A : \mathsf{U}\, m$. Cumulativity applies to the whole type and not under a binder: a function into $\mathsf{U}\, l$ is not silently a function into $\mathsf{U}\, m$, and one writes $\lambda b.\, T\, b$ when needed;
- a sum declared without a universe, like `N` above, is *small*: it lies in every universe. A sum may also be declared at a given level, `data D : U l = ...`.

**Products over levels and constraints.** A type may be polymorphic in a level: `[a] A` is the type of the `<a> t`, applied to a level by `t @l`. A type may also depend on a constraint: `[a < b] A`. A constraint product is introduced and eliminated silently: a term of type $[\psi]A$ is used as a term of type $A$ wherever $\psi$ holds in the current context. Both kinds of products are *large*: they lie in no universe.

**Deciding constraints.** Whether a constraint follows from those of the context is decided by the algorithm of Bezem and Coquand for join-semilattices with an inflationary endomorphism: constraints become Horn clauses, and the question is answered by computing a least model by forward reasoning [1]. The same algorithm detects *loops*, contexts whose constraints force $l < l$. A context with a loop is inconsistent, and in it every two terms are convertible. In practice this signals an error in the constraints, and the checker warns.

### 3.3 Definitional equality

Definitional equality is generated by

- $\beta$ for functions, pairs, level abstractions, projections and `split` applied to a constructor (of a sum or of a record); for a sum, the recursive results of the branch are the same `split` applied to the recursive fields;
- the computation rule of `idJ` on `refl`;
- the unfolding of definitions (but see `opaque` below);
- $\eta$ for functions and pairs.

There is no $\eta$ for records: an element `t` of a record type and `new R[ps, t.x`$_1$`, ...]` are not convertible. They are equal in `Id`, by the eliminator (Section 4.4).

A use of a definition is represented by its name together with its environment: the values of all the variables and levels in scope where it is declared (its *arguments*), not only those its body uses. The checker decides definitional equality strictly first, then by the arguments of a common definition, and only then by unfolding (Section 5.3); strict comparison is the first filter of the organisation proposed in Section 2.

Two occurrences of the same `split` are compared in the same way, by their environments. When these differ, the checker compares the branches, each in its own environment, with fresh variables for the pattern variables and for the recursive results. This is the congruence rule of the eliminator: a `split` is an eliminator whose branches bind the fields and the recursive results, and two eliminators are equal when their branches are equal under these binders. The recursive results stay fresh; replacing them by the `split` itself would be an $\eta$ rule for inductive types, which the theory does not have. The checker does not yet compare the types of the two occurrences; with that comparison the rule is exactly congruence of the typed eliminator.

Two occurrences of the same data type are also compared by their environments, and otherwise by the telescopes of their constructors. This is not congruence: it identifies `D a` and `D b` when the parameter is not used. It holds when a data type is read as the initial algebra of its description (constructor names and telescopes). Since parameters are uniform and there are no indexed families, the description determines the data type.

Taking the whole environment is a deliberate choice: it is simple and needs no analysis of the bodies. Its consequence is visible under `opaque`: an opaque definition is never unfolded, so two uses of it that differ in a variable its body does not use are different.

**Record types are compared by name.** Two record types are equal when they come from the same declaration with equal parameters; their fields are never compared. Two different declarations are different types even if their fields agree. This is the rule of the proof checker Half [7] and of Gofer, and it avoids comparing large records field by field. An anonymous `sig{...}` is a record declared at its occurrence. Dependent pairs $(x : A) * B$ are compared structurally. Elements of a record type are compared by their constructor: two elements built by `new` are equal when their parameters and fields are convertible, wherever they are written.

**The unfolding strategy.** `opaque f` declares that, from this point on, `f` is not unfolded: it behaves as a constant, though its body has not changed. `transparent f` undoes this; the two declarations are those of `cubicaltt` [6]. The strategy is a property of the checking run and not of the values: evaluation builds values in the same way whatever the strategy. Head normalisation, used both by conversion and to expose type shapes during checking, consults it. In particular

an `opaque` declaration placed after a segment applies to every instance of it. Making a definition opaque can only make fewer terms convertible, so it never makes an incorrect proof correct. The intended use is to lock a construction right after the lemmas that characterise it, as one does in informal mathematics.

# 4 The module system

## 4.1 Segments

A segment is a named list of units:

```
module Mon (A : U a) (e : A) (op : A -> A -> A) {
  twice (x : A) : A = op x x ;
  four (x : A) : A = twice (twice x) ;
} ;
```

The *parameters* of a segment are its units without a body: the telescope in the header and the parameters declared in the body. The level variables and the constraints of its body belong to it as well. A segment is checked once, where it is declared, with its parameters as unknowns.

An *instance* `Mon[N, zero, plus]` gives values to the parameters, in order. Each argument is checked against the type of its parameter, in which the earlier arguments have been substituted; a level parameter takes a level; a constraint of the segment must hold for the given levels. Members are then accessed as `Mon[N, zero, plus].four`, or through an alias `module NM = Mon[N, zero, plus] ;` as `NM.four`. Arguments may also be given by name, `Mon[A := N, ...]`.

Segments nest. A segment declared inside another sees the parameters of the enclosing one lexically, and needs no arguments for them:

```
module Outer (A : U a) {
  module Inner { k (x : A) : A = x ; } ;
  kk (x : A) : A = Inner.k (Inner.k x) ;
} ;
```

A development spread over several files is a nest of segments: each file becomes a segment (Section 6), and a file included inside a segment sees the enclosing parameters.

## 4.2 Instances share

An instance does not copy the body of the segment. It only records the values of the parameters, in an environment; the members are evaluated in that environment when they are used. A member therefore keeps a single identity, its qualified name, and two occurrences of a member are compared by this name and the values of the parameters of its environment. In particular `Mon[N, zero, plus].twice` reached through two different aliases is one object, and the equation

```
share : Id (N -> N) (Mon[N, zero, plus].twice) (NM.twice) = refl NM.twice ;
```

holds without unfolding `twice`.

The arguments of an instance are checked where the instance is written. An expression `M[a, b].x` written at several places is checked at each of them, against the types of the parameters of `M`; an alias `module MI = M[a, b] ;` checks the arguments once, and every later `MI.x` is a use of a name. When the parameter types are long, as for a lemma with many hypotheses, the difference is

visible (Section 7). This is the informal practice of introducing a notation for an object once and using the notation.

When a large development instantiates the same segment many times, as a formalisation of a model typically does (a construction made over an abstract base, then applied to each instance), this is what keeps checking fast: equal things are recognised as equal by their names.

### 4.3 Parameters stated once

Because segments nest and share, assumptions of a whole development are stated once, as parameters of one outer segment. For instance the model formalised with Unit assumes function extensionality and univalence for the metatheory; the main file reads

```
module Model (fe : [b] Axioms[b].FunExt) (univ : [b] Axioms[b].Univalence) {
  import V ;
  import Presheaf ;
  ...
} ;
```

where each `import` inserts a file as a sub-segment (Section 6), so that every file sees `fe` and `univ`. Likewise the abstract base of the model (a category with an interval and a presheaf of cofibrations) is one segment whose parameters are the data of the base; the generic constructions are imported inside it, and an instance of the base (posets, or simplicial sets) is one instance of that segment, after which every generic construction is a member of the instance. Excluded middle, used at one place, is the parameter of a separate segment, so the files that may use it are exactly those imported inside that segment.

**Changing the assumptions.** Function extensionality follows from univalence. Once this is proved, as a segment `FunExtUA` producing, from univalence at a level $b$, function extensionality at $b$, the whole development is obtained under univalence alone by one instance:

```
module ModelUA (univ : [b] Axioms[b].Univalence) {
  feU : [b] Axioms[b].FunExt = <b> FunExtUA[b].FE[univ @b].fe ;
  module M = Model[feU, univ] ;
} ;
```

Every result of the development is then available as `ModelUA[univ].M.....` Since an instance only binds the parameters, checking it checks the two arguments and nothing of the body: the development is neither copied nor checked again.

### 4.4 Records

A segment is never a type. A record is declared like a segment, with the keyword `record`:

```
record Mon (l : Level) {
  A : U l ;  op : A -> A -> A ;
  sq (x : A) : A = op x x ;
  e : A ;  sqe : Id A (sq e) e ;
} ;
```

Its parameters are those of the header; the declarations of the body without a definition are its *fields*, and its definitions are its *derived members*. A field type may use the earlier fields and derived members.

- *Formation.* `Mon[a]` is a type: all parameters are given, the fields are not. It lies in the maximum of the universes of its fields.
- *Construction.* `new Mon[a, N, plus, zero, refl zero]`, with the parameters and then the fields, is an element; `new` is the only constructor.
- *Projection.* For `M : Mon[a]`, `M.op` is a field, and `(new Mon[a, N, plus, ...]).op` computes to `plus`.
- *Derived members.* `M.sq` is the definition `sq` applied to the element `M`. Like any definition, it unfolds unless it is `opaque`, on a constructed element and on a variable alike, and two uses are equal when they apply it to equal elements.
- *Elimination.* `split { Mon A op e sqe -> b }` binds the fields (the parameters are given by the type) and computes on an element built by `new`. It proves $\eta$:

```
etaM : (M : Mon[a]) -> Id (Mon[a]) M (new Mon[a, M.A, M.op, M.e, M.sqe]) =
  split { Mon A op e sqe -> refl (new Mon[a, A, op, e, sqe]) } ;
```

A record comes with a constructor `mkR`, the function of its parameters and fields returning `new R[...]`, defined right after it; it is not generated when a field type uses a derived member, as `sqe` does here. A record may be declared inside a segment and use its parameters; its identity then includes the values of these parameters. The body of a record may `open` an instance of a segment, which may depend on earlier fields: this only gives short names to the members of the instance, which keep their identities.

**Fixed fields.** `Mon[a, N]` or `Mon[l := a, A := N]` is the type of the elements of `Mon[a]` whose field `A` is `N`. For `M : Mon[a, N]`, a projection `M.A` is read as `N` where it is checked, and the types of the later projections use `N`: `M.e : N`. A field may be fixed only together with the fields its type mentions. Fixing more fields gives a subtype: `Mon[a, N]` is a subtype of `Mon[a]`. Since the fixed value comes from the type of the projection, it is lost when an element is used at the larger type: for `pass (M : Mon[a, N]) : Mon[a] = M`, `(pass M).A` is not convertible to `N`, though the two are equal in `Id`, by the eliminator.

# 5 The checker

The checker is written in Haskell, under 6000 lines, with no library beyond the base. It follows the standard architecture of evaluation into values with closures, conversion on values, and bidirectional type checking [5, 13]. This section gives what is needed to implement it; the choices that differ from the usual ones are the representation of definitions, the order of the steps of conversion, and the checking of a list of units.

## 5.1 Syntax after resolution

The parser produces terms with names; a resolution pass, before any checking, decides from scope alone what each name denotes, and the core sees only resolved terms. Every declaration (definition, parameter, segment, instance, record) gets a unique *identity*, its qualified name, and every use of a

declaration is written with that identity. Bound variables may not shadow a variable in scope (as in [13]), so substitution never captures, and every binder ($\lambda$, $\Pi$, $\Sigma$, pattern variable) is renamed to a name unique in the program: the binder then identifies its occurrence in the source, hence its body. Each occurrence of a `split`, of a `data` body, and of an anonymous `sig` likewise gets an identity. The same pass checks the sequential scope and the strict positivity of the sums; the filling of omitted arguments (Section 6.1) also records, in each branch of a `split`, the positions of the recursive fields.

### 5.2 Values

Terms are evaluated into values in an *environment*, a list of frames of three kinds: a variable bound to a value, a level variable bound to a level, and a *group* of consecutive definitions, segments and instances, entered at once and not evaluated. The *dynamic part* of an environment is the list of values of its variables and levels; groups contribute nothing to it. The values are:

| | |
|---|---|
| $x_k$ | a generic variable, created when checking under a binder |
| $c$ | a parameter of the program, or of a segment being checked: a constant |
| $\langle f;\rho\rangle$ | a use of the definition $f$ in the environment $\rho$ of its group |
| $n\,v$, $n.1$, $n.l$, $\mathsf{J}(\ldots,n)$, … | eliminations of a value $n$ that is not canonical |
| $\lambda(x.t;\rho)$, $\Pi(A,x.B;\rho)$, … | closures: a binder occurrence, its body, an environment |
| $c\,\vec{v}$, $\langle D;\rho\rangle$, $\langle s;\rho\rangle$ | a constructor; a `data` body; a `split` |
| $\mathsf{U}\,l$, $(u,v)$, $\mathsf{Id}\,A\,a\,b$, $\mathsf{refl}\,a$, … | the other canonical forms |
| $R[\rho]$, $\mathsf{new}\,R[\rho]$ | a record type, an element built by `new` |

A use of a definition is the closure $\langle f;\rho\rangle$, nothing more: its body is evaluated only when the definition is unfolded. A parameter is not bound in the environment, so a lookup that reaches the outermost frame returns it as a constant; inside a segment the parameters are therefore rigid, and an instance binds them.

**Evaluation** is the usual one and never unfolds a definition: looking up a definition gives $\langle f;\rho\rangle$; application of a $\lambda$-closure extends its environment; application of $\langle s;\rho\rangle$ to a constructor $c\,\vec{v}$ evaluates the branch of $c$ with its pattern variables bound to $\vec{v}$ and each recursive result bound to the same split applied to the corresponding field (to $\lambda\vec{y}.\,s\,(v_j\,\vec{y})$ for a function field); every other elimination of a value that is not canonical builds a neutral value; $\mathsf{J}$ computes on $\mathsf{refl}$. A segment in the environment is its units, the environment where it is declared, and its identity; an instance $M[\vec{a}]$ is computed by walking the units of $M$ in order, binding each parameter to the next argument and entering each group of definitions on the way. The member $M[\vec{a}].f$ is then $\langle f;\rho\rangle$ with $\rho$ this environment, whose dynamic part consists of the arguments $\vec{a}$. A record element $\mathsf{new}\,R[\rho]$ holds the values of the parameters and fields of $R$; a derived member $t.d$ is $\langle d;\rho_t\rangle$, where $\rho_t$ binds the element $t$ itself and its fields as projections of $t$. A variable of record type is a generic variable with its record: there is no $\eta$ for records.

**Weak head normal forms.** The *folded* weak head normal form performs $\beta$ for functions, pairs, records and levels, `split` on constructors and $\mathsf{J}$ on $\mathsf{refl}$, and unfolds no definition. A value in folded

form may be *blocked* by a definition: its head is $\langle f; \rho \rangle$, or an elimination whose principal argument (the function, the pair, the scrutinee of a `split`, the path of $\mathsf{J}$) is blocked by $f$. Unfolding replaces $\langle f; \rho \rangle$ by the value of the body of $f$ in $\rho$, followed by the folded reduction this enables. The weak head normal form unfolds blocking definitions until none is left, except the opaque ones: the set of opaque identities is part of the checking context and is consulted only here.

### 5.3 Conversion

*Strict equality* $u =_{\mathrm{s}} v$ is structural equality of values: two uses $\langle f; \rho \rangle$, $\langle f'; \rho' \rangle$ are strictly equal when $f = f'$ and the dynamic parts of $\rho$ and $\rho'$ are strictly equal; two closures when they have the same binder occurrence and strictly equal dynamic parts; two `split`s, `data` bodies or named record types when they are the same occurrence with strictly equal dynamic parts. It unfolds nothing and evaluates nothing. Conversion $u \equiv v$ is decided in four stages:

1. if $u =_{\mathrm{s}} v$, they are convertible;
2. if $u$ and $v$ are eliminations, with the same eliminators, of uses $\langle f; \rho \rangle$ and $\langle f; \rho' \rangle$ of one definition, and the dynamic parts of $\rho, \rho'$ and the arguments of the eliminations are pairwise convertible, they are convertible;
3. both are put in folded weak head normal form; then, as long as one is blocked by a non-opaque definition, a *head step* is made and strict equality is tested again. If only one side is blocked, it is put in weak head normal form at once. If both are, by definitions $f$ and $g$, the one declared later is unfolded once, since only its unfolding can reach the other; if they are the same, or not ordered, both are;
4. when neither side is blocked, they are compared by their outer form, recursively with conversion: $\Pi$ and $\Sigma$ by their domains and their codomains at a fresh variable; $\lambda$ by $\eta$ (both sides applied to a fresh variable), pairs by $\eta$; constructors, $\mathsf{Id}$, $\mathsf{refl}$ and neutral eliminations componentwise; named record types by occurrence and dynamic parts (never by fields); elements of records by their record, parameters and fields; universes and levels by the decision procedure for levels; two uses of an opaque definition by their dynamic parts; `split`s and `data` bodies as described in Section 3.3.

Stage 2 is where sharing pays: two uses of a lemma, or of a member of one instance, are compared through their arguments without opening the lemma. Opening only the later definition in stage 3 matters for definitions stacked on each other, where unfolding both sides in lock step would expand the earlier one needlessly.

### 5.4 Bidirectional checking

Checking has two judgments, $\Gamma \vdash t \Leftarrow A$ (check $t$ against the value $A$) and $\Gamma \vdash t \Rightarrow A$ (infer the type of $t$). The context $\Gamma$ holds the environment, the types of the variables and definitions in scope, a counter for fresh variables, the level variables and constraints, and the set of opaque definitions.

- Names, applications, projections, level applications, $\mathsf{J}$ and members of instances are *inferred*; to check them against $A$, their type $A'$ is inferred and $A' \leq A$ is tested, where $\leq$ is conversion except at the top, where $\mathsf{U}\, l \leq \mathsf{U}\, m$ when $l \leq m$ and a record type with more fixed fields is below one with fewer. An inferred type is kept as it is computed, with its definitions folded, so that $A' \leq A$ starts with the strict and by-name stages.
- The introduction forms ($\lambda$, pairs, $\mathsf{refl}$, constructors, `split`, `struct`, types) are *checked*: the expected type is put in weak head normal form, and its outer form decides the rule. A `split`

is checked against $(x : D) \to P\,x$ with $D$ a sum: for each constructor $c$, its pattern variables are fresh variables of the types of the fields, its recursive results fresh variables of type $P\,x_j$ (or $(\vec{y} : \Psi) \to P\,(x_j\,\vec{y})$), and the branch is checked against $P\,(c\,\vec{x})$.

- To infer $f\,a$, the type of $f$ is inferred and put in weak head normal form; it must be $\Pi(A, x.B; \rho)$; then $a \Leftarrow A$ and the type is $B$ in $\rho$ extended by the value of $a$. Expressions are evaluated in the environment of $\Gamma$ as soon as their values are needed in a type.
- A constraint product $[\psi]A$ is discharged silently: an expected type $[\psi]A$ adds $\psi$ to the context, and an inferred type $[\psi]A$ becomes $A$ once $\psi$ is valid.

### 5.5 Units and segments

A list of units is checked from left to right, threading the context:

- `level a` and `constraint` extend the level variables and constraints;
- a parameter `x : A`: $A$ is checked to be a type, and $x$ is added with type $A$, but not bound: it is a constant;
- a group of consecutive definitions is entered at once: the environment receives the group, the context the types; then each definition is checked, its type as a type and its body against it;
- a segment `module M (...) { ` $\vec{u}$ `}` is checked once: its units are checked in the current context, with its parameters as constants; then the segment (its units, the current environment, its identity) is added to the environment;
- an instance `module N = M[`$\vec{a}$`]` walks the units of $M$ as in evaluation, checking each argument against the type of its parameter in the environment built so far, and each constraint of $M$ for the given levels; the result is added to the environment. An expression $M[\vec{a}].x$ written inside a term is checked in the same way, at the place where it is written;
- `opaque f` and `transparent f` change the set of opaque definitions for the rest of the list.

The context after a unit does not depend on whether the unit checked: definitions enter the context by their types and bodies, not by their proofs of correctness. Hence the result of each declaration can be computed separately, with a time budget (ten seconds per declaration; an overrun counts as a failure), and one file can be checked while the others are trusted. The checker also runs as a server that loads a development once and, after an edit, checks only the files whose content changed and those depending on them; the 210 files of the model (about 12000 declarations) check from scratch in about 54 seconds, and each file separately, the others trusted, in under five seconds.

**Measuring the work of conversion.** With the option `-opened`, the checker reports for each declaration how its comparisons were settled (strictly, by the arguments of a common definition, or by unfolding) and which definitions were unfolded, with how often. With `-opens F`, where the file `F` lists for each declaration the definitions its informal proof uses, it reports any other definition it had to unfold. A definition unfolded outside that list is a step the text leaves implicit, or work the checker repeats. These counts are the current, partial, form of the residual list of Section 2: they show which definitions a comparison needed, not yet the comparison itself.

Moving to recursion through eliminators changed little in the model: of 174 recursive definitions, 161 were rewritten mechanically, a call `f m` becoming the recursive result for `m`; 12 helpers that were mutually recursive with their caller now take the recursive result as an argument; and one

proof (equality of lifts of iterative sets) was restated as a recursion on one argument. Checking time did not change. A hidden argument whose filling failed, and a recursive result whose positions were not recorded, are errors when used, so that they make no equality in a run that trusts the definition.

*Remark* 1. What Unit does not have: no unification and no implicit arguments beyond the matching of Section 6.1; no indexed inductive families; no recursion by name (only primitive recursion, through the eliminators); no mutual, nested or indexed inductive types; no pattern matching on `refl`; no proof irrelevance. These omissions are deliberate: each would make checking less predictable, and the formalisation did not need them.

# 6 Notation read before checking

The following conveniences are not part of the core language: they are translated into core terms before checking, and the core never sees them. This preliminary syntactic processing also checks the restriction on local units in Section 3.1.

## 6.1 Omitted arguments

A parameter written in braces, `f {x : A} (y : B) : C`, is *hidden*: the type of `f` is the ordinary $(x : A) \to (y : B) \to C$, and a use `f u` is read as `f _ u`, where `_` is a hole. A hole may also be written explicitly anywhere, and a hidden argument may be given explicitly as `{e}`. Holes are filled before the term is checked, by one-way *matching*, never by unification:

- the inferred type of each explicit argument is matched against the corresponding domain, with the holes as unknowns; two applications are matched argument by argument only when they have the same head; otherwise both are computed and matching continues on their head normal forms;
- the expected type only fills holes left open by the explicit arguments;
- two different solutions for one hole are an error, so the result does not depend on the order;
- a solution is written back as a term in the current scope, using names only.

A hole that is not determined this way is an error that asks for an explicit `{e}`. This works well for lemmas about paths, such as `transH p (symH q)`, whose endpoints are determined by the explicit arguments, and it is deliberately not clever.

## 6.2 Equational chains

The notation

```
calc A { a0 =< p1 > a1 =< p2 > a2 ... =< pn > an }
```

proves $\mathsf{Id}\, A\, a_0\, a_n$. It stands for the right-nested composite of the paths $p_i$; each $p_i$ is checked against $\mathsf{Id}\, A\, a_{i-1}\, a_i$, so every point is written once and an error is reported at its step.

## 6.3 Scoping

`open M ;` (or `open M[args] ;`) makes the members of a segment available by their short names. The resolver replaces every short name by the qualified name it stands for, `M[args].x`, so the core only sees qualified names. For a variable `c` of a record type, `open c` gives short names to `c.x`.

`import F ;` is expanded when the files are read: the file `F.seg` is inserted as `module F { ...} ;`, once. A `where` block is a `let`, sequential like every list of units: its definitions see the earlier ones and the parameters of the definition, but not the definition itself. The restriction on local units in Section 3.1 applies here too: in particular a `where` block cannot introduce a data type, a recursive definition or a module. The constructor `mkR` of a record (Section 4.4) is an ordinary definition added by the resolver after the record. The resolver also separates segments from records: a segment used as a type, `new` applied to a segment, or a member of a record reached without an element, is an error.

# 7 A case study: cohomology in synthetic Stone duality

*Cohomology in synthetic Stone duality* [3] extends synthetic Stone duality [12] with cohomology in all degrees, with coefficients in families of abelian groups over a compact Hausdorff space. It uses Eilenberg–MacLane spaces as constructed in homotopy type theory by Wärn [8]. The formalisation in Unit covers the paper except one proposition that needs $n$-truncations of arbitrary types, together with the parts of [8] it uses. Its main result is Theorem 3.3.2 of [3] as stated: for $X$ compact Hausdorff, a Stone space $S$ with a surjection onto $X$, and every family $B$ of overtly discrete abelian groups over $X$,

$$H^n(X, B) \;\cong\; \check{H}^n(X, S, B)$$

as groups, naturally in $B$, for every $n$. Here $H^n(X, B)$ is the fundamental group of $(x : X) \to K(B\,x, n+1)$, which is the set truncation of $(x : X) \to K(B\,x, n)$; with this choice every map between cohomology groups is the fundamental group of a pointed map, and is a homomorphism for that reason. The proof is by induction on $n$, through the long exact sequences of both cohomologies for a short exact sequence of coefficients and the naturality of their connecting maps.

**Organisation.** The development consists of 91 files of its own, in three directories: the topological part (Scott continuity of Stone spaces, the Barton–Commelin axioms, the unit interval), Eilenberg–MacLane spaces and Čech complexes in plain homotopy type theory (no axiom of synthetic Stone duality), and the cohomology of compact Hausdorff spaces. The files of synthetic Stone duality are reused, not copied: a directory refers to the files of another, and a file is checked once in each development importing it. The whole, about 8500 declarations in 172 files, checks from scratch in under four seconds. The axioms of synthetic Stone duality form one record, a parameter of the segments that use them; the files of plain homotopy type theory are those whose segments do not take it.

The formalisation was written by an LLM working from the paper, in two parallel sessions, one per part, with a mathematician deciding the scope and the form of the main statements. The two parts met through a file listing, for each result one part used from the other, its statement in Unit and its declaration: the boundary between the parts is a list of mathematical statements.

**Openings.** The measure of Section 5 was recorded for every declaration, with a guideline of about 500 unfoldings for one declaration. When the main theorem was first proved, 16 declarations were above it, although each checked in a fraction of a second. Each case had one of three causes, all of them mismatches between the formal text and the informal argument, and each was removed without changing any statement:

- *An instance written repeatedly.* The induction step used the cokernel lemma, a segment with twenty hypotheses, at two short exact sequences, and wrote the instance at each use; each

use checked the twenty arguments again. Naming the induced isomorphism once, inside the step, brought the naturality proof from 1622 unfoldings to 491 (Section 4).

- *One notion under two names.* An abbreviation such as $\Omega K(A, m+1)$ for the identity type $\mathsf{Id}\,(K(A, m+1))\,\mathrm{pt}\,\mathrm{pt}$, used next to lemmas stated with the identity type, is unfolded at every comparison of the two. Writing the notion one way brought the proof that $K(p) \circ K(i)$ is null from 1333 unfoldings to 475.
- *A lemma stated in other terms than the lemmas it uses.* That the delooping of a homomorphism $h$ is multiplicative on loops is, informally, the remark that on loops it is $\mathrm{band}^{-1} \circ h \circ \mathrm{band}$, followed by the fact that the band is a homomorphism. Stating the first point as a lemma, proved by reflexivity, and the second in the terms of the band, replaced one proof of 1576 unfoldings by two of 240 and 490.

Two declarations remain above the guideline, at 513 and 722. The total checking time hardly changed (from 4.0 to 3.8 seconds): the counts did not measure a performance problem, but places where the formal text did not follow the informal argument. This is the correspondence between the work of the checker and the steps of a proof that Section 2 proposes to make systematic.

## References


[1] M. Bezem and T. Coquand. Loop-checking and the uniform word problem for join-semilattices with an inflationary endomorphism. *Theoretical Computer Science* 913, 2022.

[2] M. Bezem, T. Coquand, P. Dybjer and M. Escardó. Type theory with explicit universe polymorphism. arXiv:2212.03284, 2022 (revised and extended version, 2024).

[3] F. Cherubini, T. Coquand, F. Geerligs and H. Moeneclaey. *Cohomology in synthetic Stone duality.* Manuscript, version of 24 October 2026.

[4] R. M. Burstall. Inductively defined functions in functional programming languages. *Journal of Computer and System Sciences* 34(2/3):409–421, 1987.

[5] T. Coquand. An algorithm for type-checking dependent types. *Science of Computer Programming* 26(1–3):167–177, 1996.

[6] C. Cohen, T. Coquand, S. Huber and A. Mörtberg. Cubical type theory: a constructive interpretation of the univalence axiom. In *TYPES 2015*, LIPIcs 69, 2018. Implementation `cubicaltt`: `https://github.com/mortberg/cubicaltt`.

[7] J. Cederquist. *A Pointfree Approach to Constructive Analysis in Type Theory.* PhD thesis, Chalmers University of Technology and University of Göteborg, 1997. See Section 2 of the third paper, *An implementation of the Heine–Borel covering theorem in type theory*, and Section 7.1 of the final paper for descriptions of Half.

[8] D. Wärn. *Eilenberg–MacLane spaces and stabilisation in homotopy type theory.* arXiv:2301.03685, 2023.

[9] D. Hilbert. Über das Unendliche. *Mathematische Annalen* 95:161–190, 1926. Lecture given in 1925.

[10] F. Cherubini, T. Coquand and M. Hutzler. *A foundation for synthetic algebraic geometry.* arXiv:2307.00073, 2023.

[11] F. Cherubini, T. Coquand, M. Ritter and D. Wärn. *Projective space in synthetic algebraic geometry*. arXiv:2405.13916; manuscript version of 19 October 2026.

[12] F. Cherubini, T. Coquand, F. Geerligs and H. Moeneclaey. *A foundation for synthetic Stone duality*. arXiv:2412.03203, 2024.

[13] Q. Wang. *A Haskell Implementation for a Dependent Type Theory with Definitions*. Master's thesis, Chalmers University of Technology and University of Gothenburg, 2021.